Homework 11 — STAT 432 (Fall 2026)

Name: Owen Park **NetID:** owenp3

Everything is in Python (numpy, pandas, matplotlib, scipy for Question 3 only). The one random seed is **43211**, passed to `numpy.random.default_rng`. Every random draw in the homework (the 10 observations, all initial assignments in Questions 1 and 2) comes from that one generator in the order the cells run. K-means in Questions 1 and 2 is my own code below, no library K-means.

Question 1: Implementing K-means

I draw the 10 points, then make a random starting assignment to $K = 2$ clusters. To guarantee neither cluster is empty, the first K labels are one of each cluster and the rest are random, and then the whole vector is shuffled. The helper `cluster_means` keeps the old mean for a cluster that ends up empty, which is the rule part (b) asks for.

In [1]:
```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
pd.set_option("display.width", 130)
OI = ["#0072B2", "#E69F00", "#009E73", "#D55E00", "#CC79A7", "#56B4E9", "#F0E442", "#000000"]

rng = np.random.default_rng(43211)                # the one seed for the whole homework
X = rng.standard_normal((10, 2))                   # 10 observations, two independent N(0, 1) coordinates

def random_assignment(n, K, rng):
    """Random labels 0..K-1 with every cluster nonempty (first K slots get one of each label)."""
    lab = np.concatenate([np.arange(K), rng.integers(0, K, n - K)])
    return rng.permutation(lab)

def cluster_means(X, lab, K, old=None):
    m = np.zeros((K, X.shape[1])) if old is None else old.copy()
    for k in range(K):
        if np.any(lab == k):                          # an empty cluster keeps its previous mean
            m[k] = X[lab == k].mean(axis=0)
    return m

def closest(X, m):
    d2 = ((X[:, None, :] - m[None, :, :]) ** 2).sum(axis=2)   # n x K squared Euclidean distances
    return d2.argmin(axis=1)

lab0 = random_assignment(10, 2, rng)
print("initial assignment:", lab0)
```

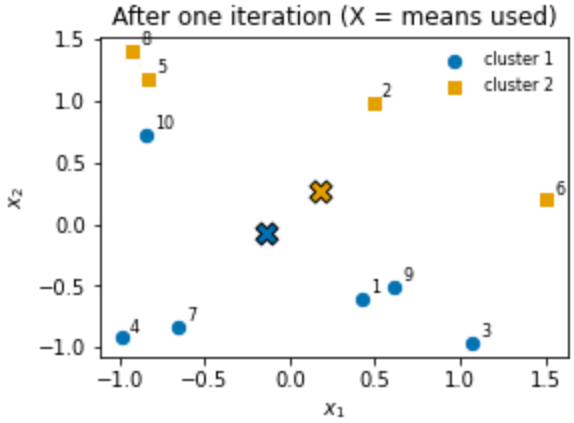
```
initial assignment: [1 1 0 0 1 0 0 0 1 0]
```

(a) One iteration. The two means from the random start are close together near the origin, because a random split mixes points from everywhere. After one reassignment step the labels change for 4 of the 10 points (observations 1, 6, 8 and 9).

In [2]:
```
m1 = cluster_means(X, lab0, 2)
lab1 = closest(X, m1)
print("cluster means from the initial assignment:\n", m1.round(3))
print("updated assignment:", lab1)

fig, ax = plt.subplots(figsize=(4.2, 3.2))
for k in range(2):
    ax.scatter(*X[lab1 == k].T, color=OI[k], marker="os"[k], s=40, label=f"cluster {k + 1}")
    ax.scatter(*m1[k], color=OI[k], marker="X", s=120, edgecolor="black")
for i, (a, b) in enumerate(X):
    ax.annotate(str(i + 1), (a, b), textcoords="offset points", xytext=(4, 3), fontsize=8)
ax.set(xlabel="$x_1$", ylabel="$x_2$", title="After one iteration (X = means used)")
ax.legend(frameon=False, fontsize=8); fig.tight_layout(); plt.show()
```

```
cluster means from the initial assignment:
 [[-0.138 -0.067]
 [ 0.177  0.264]]
updated assignment: [0 1 0 0 1 1 0 1 0 0]
```



(b) The function. The function alternates the two steps until the labels stop changing or `max_iter` is reached, then recomputes the means from the final labels and returns the labels, means, within-cluster sum of squares and the number of iterations. Nothing in it depends on two columns or two clusters, so I reuse it unchanged for the 26,600 x 3 pixel matrix in Question 2.

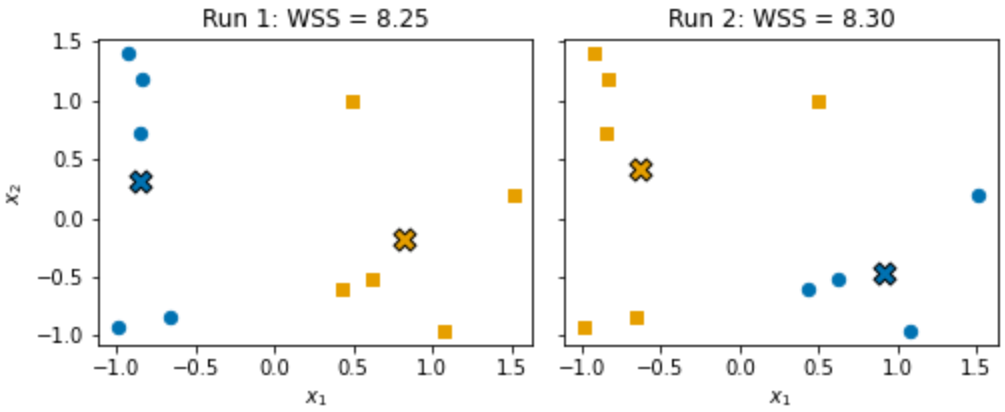
In [3]:
```
def kmeans(X, K, init, max_iter=100):
    """Lloyd's algorithm from a given nonempty assignment `init` (labels 0..K-1)."""
    X = np.asarray(X, dtype=float)
    lab = np.asarray(init).copy()
    m = cluster_means(X, lab, K)
    for it in range(1, max_iter + 1):
        new = closest(X, m)
        if np.array_equal(new, lab):                # assignments stopped changing
            break
        lab = new
        m = cluster_means(X, lab, K, old=m)
    m = cluster_means(X, lab, K, old=m)             # recompute from the final assignment
    wss = sum(((X[lab == k] - m[k]) ** 2).sum() for k in range(K))
    return {"cluster": lab, "means": m, "wss": wss, "iterations": it}
```

(c) Two random starts. Two more random starts from the same generator, each run to convergence.

In [4]:
```
runs = []
fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.1), sharex=True, sharey=True)
for r, ax in enumerate(axes):
    init = random_assignment(10, 2, rng)
    fit = kmeans(X, 2, init)
    runs.append(fit)
    print(f"run {r + 1}: start {init} -> final {fit['cluster']}  iterations {fit['iterations']}  WSS {fit['wss']:.3
    print("  means:", fit["means"].round(3).tolist())
    for k in range(2):
        ax.scatter(*X[fit["cluster"] == k].T, color=OI[k], marker="os"[k], s=40)
        ax.scatter(*fit["means"][k], color=OI[k], marker="X", s=120, edgecolor="black")
    ax.set(xlabel="$x_1$", title=f"Run {r + 1}: WSS = {fit['wss']:.2f}")
axes[0].set_ylabel("$x_2$"); fig.tight_layout(); plt.show()

# same grouping up to relabeling?
a, b = runs[0]["cluster"], runs[1]["cluster"]
print("same partition:", np.array_equal(a, b) or np.array_equal(a, 1 - b))
```

```
run 1: start [0 1 1 0 1 1 1 0 0 0] -> final [1 1 1 0 0 1 0 0 1 0]  iterations 2  WSS 8.251
  means: [[-0.849, 0.309], [0.825, -0.178]]
run 2: start [1 1 0 1 0 0 0 1 0 0] -> final [0 1 0 1 1 0 1 1 0 1]  iterations 2  WSS 8.297
  means: [[0.908, -0.47], [-0.626, 0.422]]
```




```
same partition: False
```

The two runs do not give the same answer. Both stop after 2 iterations, but run 1 ends at WSS 8.251 and run 2 at 8.297, and the partitions differ by exactly one point, observation 2 at about (0.5, 1.0). That point sits between the two groups, so whichever side its cluster starts on, the mean it is attached to stays closer and it never moves. Each run is a fixed point of the algorithm, but only run 1 is the better one. K-means only goes downhill from where it starts, so it finds a local minimum of the objective, not the global one. With real data I would run many random starts and keep the one with the smallest WSS.

Question 2: Image color compression

I use my own photo of a sushi platter (1560 x 1170, portrait), which has a lot of similar oranges and reds next to each other, so it is a good test of how many colors K-means needs. `thumbnail` shrinks it to fit in 200 x 200 while keeping the shape, which gives 200 x 150. Dividing by 255 puts all three channels on a 0 to 1 scale.

(a) The original is 1560 x 1170 x 3, the resized image is 200 x 150 x 3, and the pixel matrix is 30,000 x 3 (one row per pixel, columns red, green, blue).

In [5]:

```
from PIL import Image, ImageOps

img_full = ImageOps.exif_transpose(Image.open("data/sushi.jpg")).convert("RGB")   # my own photo
img = img_full.copy(); img.thumbnail((200, 200))      # keep the aspect ratio, longest side 200
A = np.asarray(img) / 255.0                           # RGB on a 0–1 scale
h, w, _ = A.shape
Xpix = A.reshape(-1, 3)
print("original:", img_full.size[::-1], " resized image:", A.shape, " pixel matrix:", Xpix.shape)
```

```
original: (1560, 1170)  resized image: (200, 150, 3)  pixel matrix: (30000, 3)
```

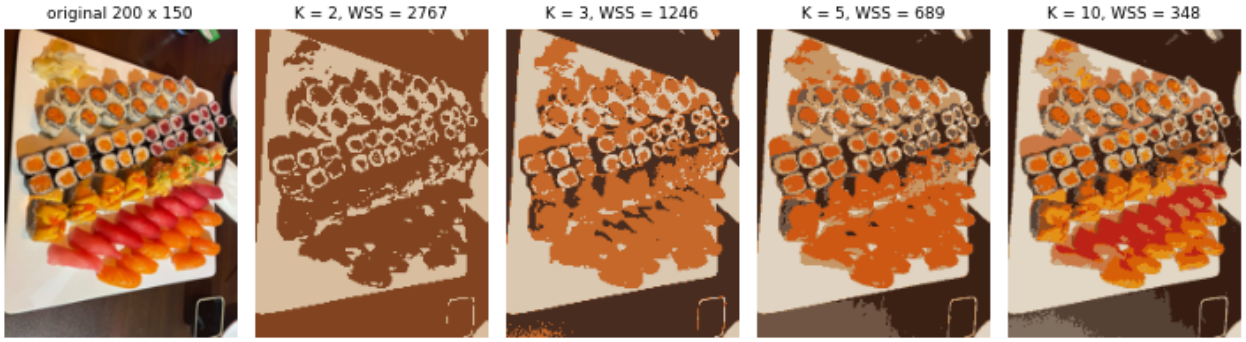
(b) Fits for K = 2, 3, 5, 10. Each fit starts from its own random nonempty assignment and runs until the labels stop changing (all four converge well before the 300-iteration cap). The printed means are colors on the 0 to 1 RGB scale, and every reconstruction is drawn on that same scale.

In [6]:

```
Ks = [2, 3, 5, 10]
fits = {}
for K in Ks:
    fits[K] = kmeans(Xpix, K, random_assignment(len(Xpix), K, rng), max_iter=300)
    f = fits[K]
    print(f"K = {K:2d}: WSS {f['wss']:8.1f}  iterations {f['iterations']:3d}  cluster sizes {np.bincount(f['cluster
    print("   means (R, G, B on 0–1):", "  ".join("(" + ", ".join(f"{v:.2f}" for v in row) + ")" for row in f["mean

fig, axes = plt.subplots(1, 5, figsize=(10, 3.6))
axes[0].imshow(A, vmin=0, vmax=1); axes[0].set_title(f"original {h} x {w}", fontsize=9)
for ax, K in zip(axes[1:], Ks):
    comp = fits[K]["means"][fits[K]["cluster"]].reshape(h, w, 3)
    ax.imshow(np.clip(comp, 0, 1), vmin=0, vmax=1)       # same 0–1 RGB scale for every panel
    ax.set_title(f"K = {K}, WSS = {fits[K]['wss']:.0f}", fontsize=9)
for ax in axes: ax.axis("off")
fig.tight_layout(); plt.show()
```

```
K =  2: WSS   2766.9  iterations  24  cluster sizes [20058, 9942]
   means (R, G, B on 0–1): (0.51, 0.27, 0.13)  (0.85, 0.75, 0.62)
K =  3: WSS   1245.7  iterations  13  cluster sizes [8022, 10536, 11442]
   means (R, G, B on 0–1): (0.86, 0.79, 0.69)  (0.28, 0.17, 0.12)  (0.77, 0.41, 0.17)
K =  5: WSS    689.3  iterations  33  cluster sizes [5963, 7592, 7601, 3956, 4888]
   means (R, G, B on 0–1): (0.88, 0.83, 0.76)  (0.81, 0.35, 0.08)  (0.23, 0.13, 0.08)  (0.44, 0.34, 0.27)  (0.78,
0.59, 0.40)
K = 10: WSS    348.3  iterations  52  cluster sizes [1839, 1959, 1639, 2687, 3011, 5423, 6325, 2850, 1449, 2818]
   means (R, G, B on 0–1): (0.74, 0.13, 0.09)  (0.50, 0.43, 0.37)  (0.89, 0.57, 0.09)  (0.79, 0.47, 0.27)  (0.79,
0.66, 0.51)  (0.89, 0.84, 0.77)  (0.21, 0.11, 0.07)  (0.34, 0.26, 0.21)  (0.59, 0.30, 0.12)  (0.84, 0.37, 0.03)
```



(c) Choice of K. I would pick $K = 10$, even though the WSS curve alone points lower. The WSS goes 2,767 → 1,246 → 689 → 348, so the saving per extra cluster drops from about 1,521 (2 to 3) to 278 (3 to 5) to 68 (5 to 10), and an elbow rule would stop at 5. But at K = 5 the tuna is gone: the red nigiri and the salmon share one orange mean, so the picture loses the main contrast on the plate. K = 10 is the first fit with a real red cluster (mean about (0.74, 0.13, 0.09), 1,839 pixels), and it splits the oranges into three shades instead of one. For this image the colors that matter cover a small share of the pixels, so the WSS, which is dominated by the large plate and table areas, undersells them.

Question 3: Hierarchical clustering of cars

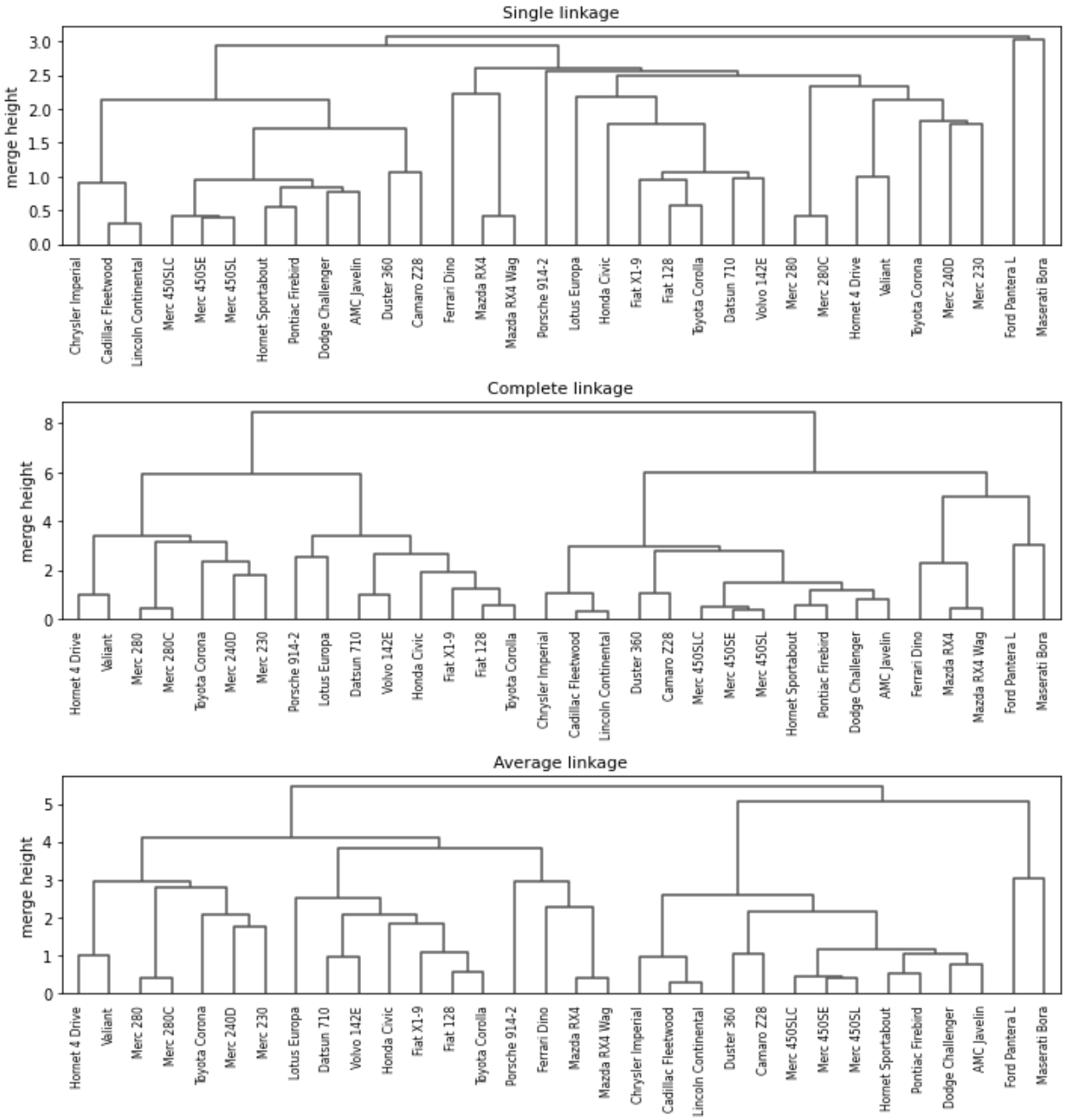
Each of the 11 variables is standardized with its own mean and SD, and `car` is only used for labels. I use scipy's `linkage` with Euclidean distance for all three methods.

In [7]:

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

cars = pd.read_csv("data/mtcars.csv")
names = cars["car"].tolist()
V = cars.drop(columns="car")
Z = (V - V.mean()) / V.std()                          # mean 0, SD 1 for each of the 11 variables

fig, axes = plt.subplots(3, 1, figsize=(10, 10.5))
links = {}
for ax, method in zip(axes, ["single", "complete", "average"]):
    links[method] = linkage(Z.to_numpy(), method=method, metric="euclidean")
    dendrogram(links[method], labels=names, ax=ax, leaf_font_size=7.5, leaf_rotation=90, color_threshold=0, above_t
    ax.set_title(f"{method.capitalize()} linkage", fontsize=11); ax.set_ylabel("merge height")
fig.tight_layout(); plt.show()
```



(a) Comparing the linkages. Single linkage chains. Most cars join one long staircase at low heights, and the Ford Pantera L and Maserati Bora hang on at the very top as a separate pair, so it does not give usable groups. Complete linkage gives the most balanced tree: a small, light-car branch on the left and a branch on the right that splits into the big American V8s and a sporty group (Mazda RX4 pair, Ferrari Dino, Pantera, Bora). Average linkage sits in between. It finds the same V8 block, but it leaves Pantera and Bora as their own late branch and puts the Ferrari Dino and the Mazdas with the small cars.

In [8]:
```
K3 = 3
cl = fcluster(links["complete"], K3, criterion="maxclust")
for k in range(1, K3 + 1):
    print(f"cluster {k} ({(cl == k).sum()} cars):", ", ".join(np.array(names)[cl == k]))
tab = V.groupby(cl).mean().round(2)
tab.insert(0, "n", np.bincount(cl)[1:])
tab
```

```
cluster 1 (15 cars): Datsun 710, Hornet 4 Drive, Valiant, Merc 240D, Merc 230, Merc 280, Merc 280C, Fiat 128, Honda
Civic, Toyota Corolla, Toyota Corona, Fiat X1-9, Porsche 914-2, Lotus Europa, Volvo 142E
cluster 2 (12 cars): Hornet Sportabout, Duster 360, Merc 450SE, Merc 450SL, Merc 450SLC, Cadillac Fleetwood, Lincol
n Continental, Chrysler Imperial, Dodge Challenger, AMC Javelin, Camaro Z28, Pontiac Firebird
cluster 3 (5 cars): Mazda RX4, Mazda RX4 Wag, Ford Pantera L, Ferrari Dino, Maserati Bora
```

Out[8]:

	n	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
1	15	24.65	4.53	131.65	91.33	3.90	2.58	19.16	0.93	0.53	3.93	1.80
2	12	15.05	8.00	357.62	194.17	3.12	4.10	17.14	0.00	0.00	3.00	3.08
3	5	18.50	6.80	223.40	198.80	3.84	3.00	15.62	0.00	1.00	4.60	5.20

(b) Chosen clusters. I chose complete linkage with 3 clusters. The biggest gap in the complete dendrogram is between the last merge (8.48) and the one below it (6.02), which by itself says 2 clusters. But that cut puts the five sports cars in with the heavy V8s, and the table shows they are very different cars, so I cut one level lower, at about 5.95. The gap there is small (5.90 to 6.02), so the dendrogram alone does not decide between 3 and 4. The fourth cluster would only split the small cars into older sedans and tiny imports, so I stop at 3. The 15 cars in cluster 1 are small economy cars: about 24.7 mpg, 4.5 cylinders, 91 hp and 2.6 thousand lb, mostly straight engines (vs 0.93). The 12 cars in cluster 2 are heavy V8s: 15.1 mpg, 8 cylinders, 358 cubic inches, 4.1 thousand lb, all automatic with 3 gears. The 5 cars in cluster 3 are sports cars: about 199 hp like the V8s but only 3.0 thousand lb, all manual with about 4.6 gears and 5.2 carburetors, and the fastest quarter mile (15.6 s). Standardizing matters because the raw variables live on very different scales. `disp` ranges over hundreds and `hp` over hundreds while `am` and `vs` are 0 or 1, so without it the Euclidean distance would be almost only displacement and horsepower.